

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Railway Reservation and Dynamic Waiting List Management System Using Data Structures in C

Student / Team Information

Team Name: Team #	Team ID: DSCPP-III-2026-T233 Semester 3 – Section ML-2/ML-3/ML-4 Domain: Data Structures / C++
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Yash – 2510370228 yashc55581@gmail.com 
Team member 2 (Last Name, name: student ID: email, picture):	Singh, Tilak – 2510370274 tilaksingh97522@gmail.com 
Team member 3 (Last Name, name: student ID: email, picture):	Rautela, Lavya Pratap Singh – 2510370289 lavyarautela@gmail.com 
Mentor	Mr. Kartikey Arora

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Most railway reservation systems are built around the idea of a confirmed booking, and they give little attention to what happens once every seat has been taken. In practice, though, this is where the real difficulty begins: operators still need a fair and efficient way to hold waiting passengers, apply priority rules, move people up the list as cancellations come in, and keep an accurate, searchable record of everyone involved. Handling this manually is slow and error-prone, and it becomes worse as the number of passengers grows. Our project addresses this gap directly. We are building a system that manages bookings, cancellations, waiting-list movement, priority-based allocation, passenger search, an undo option for the most recent cancellation, and daily reporting, all within a single, coherent application. The people who stand to benefit are the passengers themselves, the reservation operators who currently do this work by hand, and the administrators who oversee bookings and cancellations day to day. From an academic standpoint, the project is also valuable to students and faculty as a demonstration of how classical data structures solve a genuinely practical problem. The expected outcome is a more organised, efficient, and transparent reservation process, where cancelled seats are reassigned automatically and fairly instead of being managed through ad-hoc, manual intervention.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Existing railway reservation systems are largely designed around the confirmed-booking case: a passenger books a seat, and if one is available, the transaction ends there. When demand exceeds capacity, most systems fall back on a simple, often manually maintained waiting list that does not account for priority rules, does not reassign freed seats automatically, and offers no straightforward way to search, undo a mistaken cancellation, or generate a clear daily summary. As a result, waiting-list management, seat reallocation, and passenger tracking still depend heavily on manual effort, which is slow and prone to error as the volume of passengers increases.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The project's central goal is to build a railway reservation system with dynamic, priority-aware waiting-list management. This includes implementing booking, cancellation, automatic seat allocation, and priority-based booking, and applying an appropriate data structure to each of these operations – specifically Queues, Priority Queues, Stacks, Binary Search Trees, Hash Tables, sorting algorithms, arrays, and file handling. The system will be evaluated against realistic booking, cancellation, and waiting-list scenarios to confirm that it behaves correctly and scales sensibly, and the finished project will also demonstrate its practical applicability and potential for future extension.

We have organised the work into three milestones. Phase I covers the proposal and overall design, including the problem statement, the mapping of data structures to system operations, and the initial workflow and architecture, which is the stage documented in this report. Phase II covers development and integration, where the individual modules – booking, cancellation, search, and reporting – are implemented and combined into a working system. Phase III covers final implementation, thorough testing, and a live demonstration of the completed prototype.

Project Approach (3 pts)

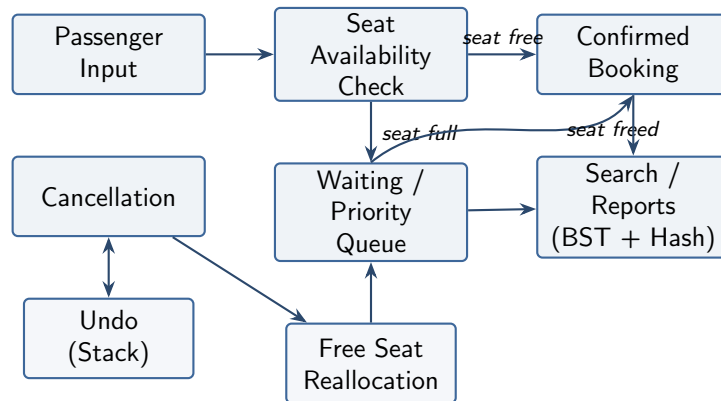
(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

Our approach begins by collecting passenger and booking details and checking seat availability. If a seat is free, the booking is confirmed immediately; if not, the passenger is inserted into a dynamic, FIFO waiting list implemented with a Queue, with a Priority Queue used to honour priority-based booking rules. When a booking is cancelled, the system processes the cancellation and automatically allocates the freed seat to the next eligible waiting passenger, and a Stack is used to support undoing the most recent cancellation if needed. A Binary Search Tree together with a Hash Table is used for fast passenger lookup, and sorting routines and file handling are used to store records and generate daily reports. In short, the core idea driving every design decision is: problem, appropriate data structure, efficient operation.

On the technology side, the system is implemented in C, using file handling for persistent data storage and the core algorithms and structures listed above – Queue, Priority Queue, Stack, BST, Hashing, and Sorting. Development is carried out in VS Code, Code::Blocks, or Dev-C++, with Git and GitHub used for version control. C was chosen specifically because it lets us demonstrate pointers, arrays, dynamic memory management, data structures, algorithms, and file handling directly and transparently, without a higher-level language hiding those mechanics. Our development methodology follows six stages: requirement analysis, system design, data-structure selection, implementation, testing with realistic reservation scenarios, and finally evaluation and documentation.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



A passenger's request first goes through a seat-availability check. If a seat is free, the booking is confirmed immediately; if not, the passenger is placed in the Waiting / Priority Queue and is promoted to a confirmed booking as soon as a seat becomes free. A cancellation triggers free-seat reallocation, which feeds the freed seat back into the waiting list, and can be undone using a Stack if the cancellation was made in error. Confirmed bookings, the waiting list, and cancellation records all feed into the Search / Reports module, which uses the BST and Hash Table for fast lookup and file handling to generate daily reservation, waiting-list, and cancellation summaries.

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

By the end of the project we expect to deliver a working railway reservation prototype with dynamic, priority-based waiting-list allocation, built entirely on the data structures introduced in this report. The prototype will demonstrate meaningful, coordinated use of arrays, queues, priority queues, stacks, binary search trees, hash tables, and sorting, and it will be able to generate daily reservation and cancellation reports from stored records. Beyond the core deliverable, the project is designed with future scalability in mind, and could reasonably be extended with a graphical interface, a proper database back end, or a web/mobile front end.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

We assume a single train (or a small, fixed set of trains) with a known, fixed seat capacity for the purposes of this prototype, and that all bookings, cancellations, and reports are processed for a single reservation day at a time. Passenger and booking data are assumed to be entered through a text-based console interface and stored locally using file handling rather than a networked database. Priority rules for the Priority Queue are assumed to follow the categories defined by the project team (for example, senior citizens or pre-booked categories) rather than external, real-world railway policy.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Course material and concepts from the *Data Structures in C* course.
2. Standard C programming and file-handling documentation.
3. Standard textbook references on arrays, queues, priority queues, stacks, binary search trees, hashing, and sorting algorithms.
4. Project-defined priority rules and test scenarios developed by the team for evaluation purposes.